

TradeVision

Operating & User Manual

Everything you need to configure, run, and use the software —
credentials, procedural workflow, architecture running guide, and expected outputs

Version 2.0 · September 2026

Table of Contents

1. [Part A — Before You Start \(Prerequisites\)](#)
2. [Part B — Installation & First Boot](#)
3. [Part C — Credentials & All API Keys](#)
4. [Part D — Running Procedure of the Architecture](#)
5. [Part E — User Workflow: Step-by-Step Procedure](#)
6. [Part F — Expected Output \(What You Will See\)](#)
7. [Part G — Full UI Options Reference](#)
8. [Part H — Operator Runbook & Safety](#)
9. [Part I — Troubleshooting & Known Limits](#)

Part A — Before You Start

| A.1 What TradeVision is

TradeVision is an Indian-equity (NSE/BSE) decision-support and **paper-trading** platform. It ingests market data and trading-view alerts, builds a deterministic intelligence packet per symbol, evaluates 8 deterministic trading rules, pushes every firing through an auditable fail-closed risk gate, and — only when you explicitly enable it — simulates fills through a paper broker. It also runs nightly per-symbol backtests and ranks strategies by demonstrated per-stock evidence.

Design fact worth repeating: by default the system acts on nothing. Rule firing, execution and

pattern analysis are all off until an operator turns them on. On a fresh install you will see *an*

empty, quiet system until you complete Part E.

| A.2 System requirements

Component	Requirement
OS	Linux (Ubuntu/Debian recommended) with Docker + Docker Compose v2
Host	8 GB RAM minimum (Postgres + Redis + 3-4 workers + backend), 20 GB free disk for images/data
Python	3.11+ (only needed if you run manage.py outside Docker)
Browser	Any modern browser (Chrome/Firefox/Edge); WebSockets for Live Activity
Time zone	Everything runs on IST (Asia/Kolkata); schedules assume India market hours

Accounts you should
have ready

See below — create them before setup so keys are ready

| A.3 Accounts you may need (create before setup)

Provider	Purpose	Create at	Required?
Zerodha Kite (or other broker adapter)	Market data + broker (paper uses none)	zerodha.com / developers.kite.trade	Only for real data/fills
Google Gemini	AI provider (default)	aistudio.google.com/apikey	For AI features only
OpenAI	AI provider (fallback)	platform.openai.com/api-keys	Optional
Anthropic	AI provider (fallback)	console.anthropic.com	Optional
DeepSeek	AI provider (fallback)	platform.deepseek.com	Optional
Ollama	Local AI (no cloud key)	ollama.com (self-host)	Optional
Marketaux	News headlines + sentiment	marketaux.com (free tier: 100 calls/day)	For news
FRED (St. Louis Fed)	Macro series ingestion	fred.stlouisfed.org/docs/api/api_key.html	For macro
Alpha Vantage / NSE API	Alternate quote/scans data	alphavantage.co / (NSE feed)	Optional
TradingView	Webhook alerts (Pine)	tradingview.com alert webhook URL	For alert ingestion
Chartink	Scan-result webhook	chartink.com scan webhook	Optional
Telegram BotFather	Drift / alert notifications	t.me/BotFather	Optional
Email (SMTP)	Transactional email	your provider (e.g. Gmail app password, SES)	Optional

Part B — Installation & First Boot

B.1 Get the code

```
git clone <your-remote> TradeVision cd TradeVision
```

B.2 Configure the environment files

Two files matter.

File	Purpose	Copy from
backend/.env (dev only)	Settings when running <code>manage.py</code> directly (bind-mounted into the backend container in dev)	backend/.env.example
infra/.env	Deployment source of truth — used by docker-compose for every container	infra/.env.example

Step 1. Edit `infra/.env` and set the non-secret essentials first:

```
DJANGO_SETTINGS_MODULE=config.settings.development # or .production for prod
```

```
DJANGO_DEBUG=True # NEVER True in production DJANGO_SECRET_KEY=<random-50-char-string> #
```

```
generate: python -c "import secrets;print(secrets.token_urlsafe(50))"
```

```
POSTGRES_DB=tradevision_db POSTGRES_USER=tradevision POSTGRES_PASSWORD=<strong-db-
```

```
password> DATABASE_URL=postgres://tradevision:<password>@pgbouncer:6432/tradevision_db
```

```
REDIS_URL=redis://redis:6379/0 MARKET_DATA_PROVIDER=zerodha # or mock for a dry run
```

Step 2. Add every API key you own from [Part C](#). Keys left empty simply disable that feature.

B.3 Start the stack

```
cd infra docker compose -f docker-compose.yml -f docker-compose.dev.yml build docker
```

```
compose -f docker-compose.yml -f docker-compose.dev.yml up -d # watch it come up docker
```

```
compose -f docker-compose.yml -f docker-compose.dev.yml ps
```

Container (dev profile)	Role
infra-backend-1	Django + Channels (ASGI on :8000) — the REST/WS API
infra-frontend-1	Vite dev server (HMR) — the web UI
infra-postgres-1, infra-pgbouncer-1	PostgreSQL + connection pooler
infra-redis-1	Redis — broker, cache, event streams
celery-worker-market	Queues: market_data, processing, intelligence, rule_engine
celery-worker-ai	Queue: ai (AI + recommendations)
celery-worker-default	Queues: notifications, analytics, default, maintenance, monitoring, webhooks (runs backtests/nightly jobs too)
celery-beat	Single scheduler instance — owns all recurring jobs

B.4 First-boot migrations & seed

```
# Apply schema + create the default account docker exec infra-backend-1 python manage.py
```

```
migrate docker exec infra-backend-1 python manage.py seed_demo_environment # optional demo
```

```
data docker exec infra-backend-1 python manage.py seed_trading_defaults # creates the 8
```

```
default strategies + rule configs
```

Step 3. Create your login account via the UI (/login → “Create an account”) or a

superuser:

```
docker exec -it infra-backend-1 python manage.py createsuperuser docker exec infra-
```

```
backend-1 python manage.py ensure_default_account # ensures a default trading account
```

```
exists for the owner
```

The nightly backtest job requires a Django user named exactly admin. Create one:

```
docker exec infra-backend-1 python manage.py create_user admin --email admin@local --
```

```
password <pw> --role owner
```

(Use the actual helper exposed by your install; otherwise `createsuperuser` with username

admin.)

B.5 Health check after boot

1. Open `http://localhost:5173` (dev) — you should see the TradeVision login screen.
2. The top-bar live dot must be **green** (connected).
3. Open `/pipeline-health` — stages should show *Healthy* once data flows.
4. Run `docker compose ps` — all workers *healthy*, beat *up*.

Part C — Credentials & All API Keys

All secrets live in `infra/.env` (and `backend/.env` in dev). They are never hard-coded and never committed — add the file to your git ignore. Values shown are *placeholders*; replace with your own keys. A key left empty disables the connected feature safely.

C.1 Core platform

Variable	Placeholder / how to get it	Effect when set
<code>DJANGO_SECRET_KEY</code>	Random 50 chars: <code>secrets.token_urlsafe(50)</code>	Signing sessions/JWTs. Required.
<code>JWT_SECRET_KEY</code>	Leave empty to inherit <code>DJANGO_SECRET_KEY</code>	JWT signing.
<code>POSTGRES_PASSWORD</code>	Strong DB password	DB auth. Required.
<code>DEFAULT_ACCOUNT_ID</code>	Leave empty → first active superuser	Default trading account.

C.2 Market data & broker

Variable	Where to get it	Notes
<code>MARKET_DATA_PROVIDER</code>	—	<code>mock</code> (synthetic) or <code>zerodha</code> .
<code>ZERODHA_API_KEY</code>	<code>developers.kite.trade</code> (Kite Connect app)	Sandbox can use the shared demo app.
<code>ZERODHA_API_SECRET</code>	Same app page	Pair with the key.
<code>ZERODHA_ACCESS_TOKEN</code>	After Kite login (or Redis hot token)	Session token; hot token in Redis overrides.
<code>ZERODHA_REQUEST_TOKEN</code>	One-time login exchange	Only during handshake.
<code>ZERODHA_PRODUCT</code>	—	MIS (intraday) or CNC.
<code>NSE_API_KEY /</code>	NSE data vendor agreement	Alternate quote/scan feed.

NSE_API_BASE_URL

ALPHA_VANTAGE_API_KEY alphavantage.co/support/#api-key Alternate quotes.
(free)

C.3 AI providers

Variable	Where to get it	Default model
AI_PROVIDER	—	gemini (priority order: claude, gemini, openai, deepseek, ollama)
GEMINI_API_KEY	aistudio.google.com/apikey	gemini-1.5-pro
OPENAI_API_KEY	platform.openai.com/api-keys	gpt-4o
ANTHROPIC_API_KEY	console.anthropic.com	claude-3-5-sonnet-20241022
DEEPSEEK_API_KEY	platform.deepseek.com	deepseek-chat
OLLAMA_BASE_URL / OLLAMA_MODEL	Legacy: http://localhost:11434 / llama3.2	Local, no key.

Tuning	Default	Meaning
AI_CONFIDENCE_FLOOR	0.55	Drop AI below this confidence.
AI_DAILY_BUDGET_USD	10.00	Daily spend ceiling.
AI_DEDUP_WINDOW_SECONDS	300	Ignore duplicate prompts within window.

C.4 News, macro, alerts, notifications

Variable	Where to get it	Effect
NEWS_API_KEY	marketaux.com (free: 100 calls/day)	News + sentiment (provider marketaux).
FRED_API_KEY	fred.stlouisfed.org/docs/api	Macro series ingestion.
TRADINGVIEW_WEBHOOK_TOKEN	Any strong token you choose	Authenticates Pine alert

		webhooks (unset ⇒ 501).
CHARTINK_WEBHOOK_TOKEN / CHARTINK_SHARED_SECRET	Choose strong values	Chartink webhook auth + HMAC.
TELEGRAM_BOT_TOKEN / TELEGRAM_CHAT_ID	t.me/BotFather	Drift/alert notifications (both required).
EMAIL_HOST* / EMAIL_HOST_USER / EMAIL_HOST_PASSWORD	SMTP provider (Gmail app password, SES, etc.)	Transactional email.
SENTRY_DSN	sentry.io project	Error monitoring (optional).

C.5 Feature switches in the same file

Switch	Default	Notes
EXECUTION_ENGINE_ENABLED	False	False ⇒ no orders are ever created.
BROKER_ADAPTER / BROKER_ENVIRONMENT	paper / sandbox	live fails Django startup until unlocked.
ALGO_REGISTRATION_ID	"	Live execution refused while empty.
PATTERN_ENGINE_ENABLED	False	Pattern detection.
MARKET_CONTEXT_SCORING_ENABLED	False	Market-context cache.
AI_ENGINE_ENABLED	False	AI recommendations path.

Part D — Running Procedure of the Architecture

D.1 Component map

[TradingView / Chartink webhooks] — [Market-data poller (Beat 60s)] — Postgres

(source of truth) | | Redis ← (broker + event streams) | Celery workers

(market/ai/default) + Beat | Backend API (Django ASGI :8000) | Frontend (Vite :5173) —

user

D.2 The event pipeline (what runs, in order)

#	Stage	Trigger	Output you can observe
1	Ingestion	Beat poller (market hours) or webhooks	Rows in DB; Raw Ingestion Events page
2	Technical analysis	Each fresh candle (warm-up gated)	<code>technical_analysis.TechnicalAnalysisCompleted</code>
3	Intelligence packet	TA completed handler	<code>intelligence.PacketBuilt</code> ; regime + quality score
4	Rule engine	Packet built → 8 rules evaluated	<code>rule_engine.RuleFired</code> (if gate allows)
5	Risk gate	Each firing → 10 fail-closed checks	<code>RiskApproved</code> / <code>RiskRejected</code> ; Risk Decisions page
6	Execution	<code>RiskApproved</code> (only if execution enabled)	Order → Fill; Orders/Positions/Trades pages
7	Observability	Every event	Audit log, Journal, dashboards, event streams

| D.3 The event bus (reliability machinery)

- **Publish:** durable DB row first → mirror to Redis stream (`events:{type}`, cap 10,000) → flag mirrored. A 30s beat re-mirrors any orphans, so Redis outages lose nothing.
- **Consume:** a 2s beat drains consumer groups and dispatches handlers; per-(event, group) idempotency.
- **Retries:** 5 attempts, backoff [1,5,30,120,600]s; acked only on terminal outcome; dead-letters are persisted and shown on Pipeline Health.
- **Reclaim:** stale pending entries auto-reclaimed after 120s (crashed workers don't lose events).
- **Audit:** a wildcard subscriber records every domain event append-only, reconstructable by correlation id.

| D.4 Celery Beat schedule (nightly and recurring jobs)

Job	Cadence	What it does
Poll market-data watchlist	60s (market hours)	Fetches bars → TA ingestion
Detect session transitions	30s	Market-open/close events
Poll event streams	2s	Drains Redis streams
Reconcile all accounts	300s	Broker vs ledger breaks
Evaluate pipeline health	30s	Stage heartbeats
Evaluate drawdown kill-switch	60s	Auto-arm on daily/weekly loss (off by default)
Sync instrument master	22:30 IST	Nightly symbol sync
Run watchlist per-symbol backtests	03:00 IST	Backtests every watchlist stock + ranks strategies
Reconcile rule expectations	04:00 IST	Validates rule verdicts vs backtests
Evaluate calibration drift	10:30 IST	Backtest-vs-live drift verdict
Ingest macro / news / chartink	configurable	Macro (14:05), news (900s), scans

(3600s)

| D.5 Queues & workers (running procedure)

Tasks route by name to queues; workers consume explicit queue lists. **If a queue has no worker, its tasks never run.** Declared queues: webhooks, signals, analysis, ai_reasoning, decisions, execution, monitoring, portfolio, analytics, notifications, maintenance, market_data.

Worker	Consumes	Missing queue symptoms
worker-market	market_data, processing, intelligence, rule_engine	No packets/rules
worker-ai	ai	No recommendations
worker-default	notifications, analytics, default, maintenance, monitoring, webhooks	No nightly backtests, audit replay, health

Two routing caveats verified in source: rule-engine tasks route to rule_engine and monitor-

broker-connection routes to default — confirm a worker actually consumes them, or these

stages stay silent.

| D.6 How to run a manual job

```
# Backtest one symbol for evidence (needs the 'admin' user) docker exec infra-backend-1
```

```
python manage.py per_symbol_backtests --symbols SBIN --days 730 # Backtest everything in
```

```
the watchlist docker exec infra-backend-1 python manage.py per_symbol_backtests # Seed the
```

```
default strategies/rules after a fresh migrate docker exec infra-backend-1 python
```

```
manage.py seed_trading_defaults # Inspect DB quickly docker exec infra-postgres-1 psql -U
```

```
tradevision -d tradevision_db -c "select count(*) from backtesting_backtestrun;"
```

Part E — User Workflow: Step-by-Step Procedure

This is the exact procedure a user follows to go from a fresh login to a running, evidence-ranked trading system.

Step 1 — Create your account and sign in. Open `/login`, choose “Create an account”, enter Full name, Email, Password. You land on the Dashboard. The top bar shows your role badge (owner/staff/viewer).

Step 2 — Set up the symbol watchlist (drives nightly backtests). Open `/watchlist` → “Add symbol” → type a ticker (e.g. SBIN) → Add. Set a group and optional price alerts (above/below). Add all stocks you care about. *Expected:* rows appear with last price/change; “Your watchlist is empty” disappears.

This watchlist feeds *backtests only*. To also get live signals you must add the same symbols to

`MARKET_DATA_POLL_WATCHLIST` in `infra/.env` as `NSE:SYMBOL,NSE:SYMBOL2` (equals

`EXCHANGE:SYMBOL`) and restart the poller: `docker compose -f ... restart celery-beat celery-`

`worker-market`.

Step 3 — Generate per-stock evidence (nightly or manual). Either wait for the 03:00 IST job, or run now: `docker exec infra-backend-1 python manage.py per_symbol_backtests --days 730`. The runner skips completed combos and resumes interrupted ones. Expect each symbol to eventually show one run per ACTIVE strategy. *Expected output:* rows in `/backtests` with status → progress → Complete, then `StrategySymbolAffinity` rows appear for gate-clearing strategies.

Step 4 — Review the per-symbol rankings. Open `/strategies` → use the affinities panel filter to pick a symbol (e.g. SBIN). You will see which strategy earned rank 1 for that stock and its expectancy, profit factor, max drawdown and trade count. A stock with no passing strategy shows an empty table — that is correct (no fabricated evidence).

Step 5 — Create the automation rules you want. Open `/rules` → “New rule”. Configure: Name, Strategy, Symbols, Condition (Price/Volume/Indicator/News/Time), Comparison, Threshold, Action (Buy/Sell/Close/Alert/Hold), Sizing (fixed/% equity/% risk), Cooldown, max executions/day. Or use the pre-seeded rule configs from `seed_trading_defaults` and enable the specific (rule, regime) combinations you accept.

Rules created by backtesting start enabled=False. A GO verdict never flips a rule live — you must explicitly enable it.

Step 6 — Watch signals and recommendations. Open /signals (rule/model signals with Accept/Reject) and /strategies/recommendations (AI recommendations with Accept/Dismiss/Backtest). Accepting is how a signal becomes executable in downstream flows.

Step 7 — Decide whether to execute (paper). Set EXECUTION_ENGINE_ENABLED=True in infra/.env and restart the backend+default worker. Keep BROKER_ENVIRONMENT=sandbox and ALGO_REGISTRATION_ID="" for paper. *Expected afterwards:* RiskApproved decisions start producing Orders (CREATED→FILLED) on the Orders page; fills appear on Positions and Trades.

Step 8 — Monitor and stay safe. Daily: /pipeline-health (all stages Healthy), /risk/decisions (approvals/rejections & reasons), /reconciliation (broker vs ledger breaks), /audit (trail). Emergency: /kill-switch to halt everything or flatten positions.

Step 9 — Learn from outcomes. Optionally keep /journal entries, review Trader Memory (/memory) which records recommendations and their outcomes, and re-run backtests whenever you add a symbol.

E.1 What the nightly automation does for you

03:00 Beat → run_watchlist_per_symbol_backtests(days=730) → per_symbol_backtests

(subprocess, isolated) → symbols = your Watchlist entries → for each symbol × ACTIVE

strategy: skip if COMPLETED, else create isolated account + run →

SymbolRankingService.rank_symbols() → StrategySymbolAffinity rows Automatically covers NEW

watchlist symbols on the next tick.

Part F — Expected Output (What You Will See)

F.1 The healthy running system

Page / surface	Expected content when working
Top bar dot	Green pulsing “connected”
Dashboard /	Equity/P&L/win-rate cards populated, recent trades & signals tables, exposure chart
Live Activity /live	Ticks/orders/fills rolling in during market hours
Portfolio	Holdings with market value + unrealized P&L (after fills)
Orders /orders	Orders with lifecycle status (CREATED→FILLED), cancel/amend working
Trades	Open + closed blotter with P&L, fees, strategy attribution
Analytics	Realized/unrealized P&L, daily ledger, performance vs benchmark, VaR
Backtests /backtests	Completed runs with return %, max drawdown, Sharpe, trades; detail page shows equity curve + monthly heatmap
Strategies → affinities	Per-symbol rank list (rank, score, expectancy, PF, maxDD, trades) for gate-clearing combos
Signals	Pending signals with confidence; Accept/Reject
Recommendations	AI cards with expected return/Sharpe/maxDD; accept → config
News	Headlines with sentiment + relevance; Link to trade / Create rule
Risk Decisions	Rows APPROVED or REJECTED with rejection code (e.g. MISSING_STOP_LOSS)
Pipeline Health	All stages Healthy; no backlog ages beyond thresholds
Reconciliation	Matched ≥ broken; net difference 0

Audit	Append-only trail of every state change with actor + duration
Journal	Entries finalized per trade/correlation with decision + outcome
Raw Ingestion Events	Incoming webhook/poll payloads marked Processed

F.2 Expected output of a backtest run

- **Status** PENDING → RUNNING (progress bar) → COMPLETED (or FAILED with reason).
- **Result cards:** Total Return %, Net P&L, Max Drawdown, Sharpe, Profit Factor, Trades, Win Rate, Avg Trade, final equity.
- **Drill-downs:** trades table, per-symbol breakdown, parameters used.
- **Validation views:** walk-forward OOS stats, edge-validation verdicts (Pass/Marginal/Fail), cost-sensitivity breakeven.
- When 30+ trades, expectancy > 0, PF > 1.3 and maxDD < 20% are all met, the strategy earns a rank for that symbol.

F.3 Expected output a stock with no fires

Some rules legitimately never fire on certain stocks (e.g. Long Momentum on a given bank stock). You will see a COMPLETED backtest with **0 trades** and no affinity row — that is correct behaviour, not a bug.

F.4 Edge-case outputs to recognise

You see...	Meaning
MISSING_STOP_LOSS on Risk Decisions	The firing had no stop for a rule that requires one — risk fails closed by design.
0 fills but orders created	Paper broker hasn't matched yet, or execution disabled so no orders at all.
Pipeline stage Down	No heartbeat within its silence budget — check that stage's worker/queue.
Dead-lettered handler on Pipeline	A handler failed 5×; look at the audit/event trace.

Health

Backtest still RUNNING for a long time Large 730-day window; resume is cursor-based so safe to interrupt.

Part G — Full UI Options Reference

Every page, every option. Organized by navigation group.

| G.1 Overview

Dashboard — /

Controls: date-range Select (Today/7D/30D/90D/YTD) · “Refresh” · “New Backtest” · “View Signals”. **Cards:** Total Equity, Day P&L, Open Positions, Win Rate, Total Trades, Profit Factor, Max Drawdown, Net Exposure (each with delta + sparkline). **Panels:** equity line chart · exposure bar chart · Recent Trades table (time, symbol, side, size, entry, exit, P&L, status) · Recent Signals table.

Live Activity — /live

Controls: symbol input (datalist) · feed tabs (All/Ticks/Orders/Fills) · Pause/Resume · Auto-scroll checkbox · Reconnect. Rows: timestamp, source, symbol, message, severity; fills add qty + price.

Portfolio — /portfolio

Controls: account select · Group-by tabs (Asset/Account/Sector) · search · “Export CSV”. **Columns:** Symbol, Name, Quantity, Avg Cost, Last Price, Market Value, Unrealized P&L, Unrealized %, Day P&L, Weight %, Allocation bar. Footer totals + “View summary”.

Portfolio Summary — /portfolio/summary

Controls: period Select (1D/7D/30D/90D/YTD/All) · account select · “Export”. **Cards:** Net Value, Total Return, Realized/Unrealized P&L, Fees, Cash, Buying Power, Margin Usage. **Panels:** allocation pie, Top Holdings, Top Winners/Losers, daily return chart.

Holding Detail — /portfolio/holding/:id

“Close position” modal: quantity, exit type (Market/Limit), limit price, “cannot be undone” checkbox. Cards: qty, avg cost, last, market value, unrealized P&L, holding period, stop distance. Panels: price chart, position history, related orders, related trades.

| G.2 Orders & Trading

Orders — /orders

Controls: status tabs (All/Pending/Partially Filled/Filled/Cancelled/Rejected) · symbol · side (All/Buy/Sell) · type (All/Market/Limit/Stop/Stop-Limit) · date range · Clear filters · “New order”.

New order modal: Symbol, Side (Buy/Sell), Order type, Quantity, Limit price (Limit/Stop-Limit), Stop price (Stop/Stop-Limit), Time in force (Day/GTC/IOC/FOK), optional note, live notional readout. **Columns:** Order ID, Time, Symbol, Side, Type, Quantity, Filled, Limit/Stop price, Status, Filled %, Actions (Cancel/View). Footer: status counts + total notional.

Order Detail — /orders/:id

“Cancel order” · “Amend order” (qty + limit) · cards (all order fields + average fill) · Executions table · Status timeline.

Trades — /trades

Status tabs (All/Open/Closed) · symbol · strategy · date range · shortcuts Open/Closed/Export. Columns: Trade ID, Opened/Closed At, Symbol, Side, Quantity, Entry/Exit price, Fees, Net P&L, P&L %, Holding Period, Strategy, Status. Footer: net P&L, fees, count, win/loss split.

Trade Detail — /trades/:id

“Close trade” modal (exit price, optional date/note) · cards incl. R-Multiple, Return on Capital, Strategy · Legs table · Strategy Context.

Trades Open — /trades/open & Trades Closed — /trades/closed

Open: symbol, strategy, min unrealized P&L, “Close all” with per-row include checkboxes. Closed: date range, strategy, side, min net P&L, Export CSV; win/loss chart, performance-by-strategy, best/worst.

Trade Export Status — /trades/exports

Format (CSV/JSON/XLSX) · date range · “Start export” · per-row Download · progress bars · Retry on failure.

| G.3 Analytics

P&L Analytics — </analytics/pnl>

Period tabs (Daily/Weekly/Monthly) · dimension (Account/Strategy/Symbol/Asset Class) · account · “Realized only” · Export. Cards: Realized, Unrealized, Total, Fees, Win Rate, PF, Avg Win, Avg Loss. Panels: cumulative chart, per-period bars, breakdown table, biggest moves.

P&L Daily — </analytics/pnl/daily>

Date range · account · tabs Cumulative/Daily · Download CSV. Table: Date, Trades, Wins, Losses, Gross/Net P&L, Cumulative, Return %, Max Drawdown.

Performance — </analytics/performance>

Compare multi-select · benchmark (None/SPY/QQQ/BTC-USD/custom) · period · “Show risk metrics” · Rebalance basis (Time/Return). Cards: CAGR, Sharpe, Sortino, Calmar, Volatility, Max DD, Beta, Alpha, Info Ratio, Treynor. Panels: normalized returns, underwater drawdown, monthly heatmap, correlations.

Risk Analytics — </analytics/risk>

Horizon (1D/10D/30D) · method (Historical/Parametric/Monte Carlo) · confidence (90/95/99%) · account · “Recalculate”. Cards: VaR, CVaR, Gross/Net Exposure, Leverage, Concentration, Beta, Drawdown, Limit Utilization. Panels: exposure chart, top concentrations, stress scenarios, limit breaches.

| G.4 Backtesting & Research

Backtests — </backtests>

Status tabs · strategy · symbol · date range · “New backtest” · “Recently viewed” toggle. Columns: Run ID, Name, Strategy, Symbols, Range, Status, Progress, Created, Duration, Trades, Return %, Max DD, Sharpe, Actions (View/Walk-forward/Edge/Cost).

New Backtest — </backtests/new>

Name · Strategy · Symbols (chips) · Date range + “Use max available” · Timeframe (1m/5m/15m/1h/4h/1D) · Initial capital (100000) · Commission/Slippage + “Include fees” · Risks (max position size, stop %, take-profit %, max concurrent) · Optimization tabs (None/Grid/Random) · “Start backtest”.

Backtest Detail — </backtests/:id>

Download report · Rerun · Delete · links to validation views. Cards: all metrics. Panels: equity curve, drawdown, monthly heatmap, trades table, per-symbol breakdown, parameters.

Walk-Forward — </backtests/:id/walk-forward>

Window size (5/10/20/50) · step (1/5/10) · anchor windows · window count · Run. IS/OOS stats, consistency summary.

Edge Validation — </backtests/:id/edge-validation>

Perturbation (None/Price/Delay/Slippage/Partial) · magnitude (Low/Med/High) · regime · OOS fraction slider · Run. Scenario verdicts (Pass/Marginal/Fail), degradation list.

Cost Sensitivity — </backtests/:id/cost-sensitivity>

Commission slider (0–50bps) · slippage (0–50bps) · spread (0–20bps) · borrow costs checkbox · Recalculate. Sortable table, breakeven stat, cost composition.

| G.5 Automation

News — </news>

Search · source multi-select · symbol · sentiment tabs · time window (1h/24h/7d/30d) · Refresh · Unread-only. Rows + “Link to trade” / “Create rule”.

Rules — </rules> & Rule Detail

List: status tabs, strategy, symbol, auto-refresh (5/15/30/60s), per-row enable toggle, New rule; columns incl. 24h trigger count. Detail options: condition tab (Price/Volume/Indicator/News/Time), comparison (Greater/Less/Crosses above/below), threshold, lookback, action (Buy/Sell/Close/Alert/Hold), sizing (Fixed/% equity/% risk), cooldown (None/1m/5m/15m/1h/1D), max executions/day, notes, Test run, Diagnostics, Recent Triggers, Version History.

Rule Executions — </rules/:id/executions>

Result tabs · date range · symbol · auto-refresh · Export CSV. Success-rate stat, outcomes chart, top errors.

Strategies — /strategies

Category tabs (All/Trend/Mean Reversion/Momentum/Value/ML) · status · search · New/Duplicate/Pause/Resume/Delete. **Affinities panel:** filter by symbol to see per-stock ranking (rank, score, expectancy, PF, maxDD, trades).

Signals — /signals

Status tabs · source (Rule/Model/Manual) · direction · symbol · min-confidence slider · auto-refresh · Accept/Reject/View source. Expiring-soon panel.

Recommendations — /strategies/recommendations

Horizon tabs · risk level · category · Refresh · Show dismissed · per-card Accept/Dismiss/Backtest.

Patterns — /patterns

Type tabs (Chart/Candlestick/Volume/Harmonic/All) · symbol · timeframe · lookback · min-confidence · Refresh · Watch/Dismiss · Chart modal.

Trader Memory — /memory

Type tabs (Preferences/Lessons/Patterns/All) · Edit profile (risk appetite, holding period, max positions, sizing, symbols, blackouts, notes) · Forget · Clear all. Outcomes table (followed vs not).

| G.6 Trading Operations & System

Watchlist — /watchlist

Add symbol · group · search · sort (Symbol/Price/Change %/Volume) · per-row alert (above/below) · Remove. Top movers chart + alerts list.

Journal — /journal

Date range · mood (Confident/Neutral/Anxious/Frustrated/Regretful) · symbol · Favorites-only · New entry (date, time, symbols, trade id, well/improve/lessons, mood, confidence 1-5, tags) · Edit/Delete/Pin.

Audit — [/audit](#)

Actor · action type (Order/Trade/Rule/Strategy/Settings/Risk/Auth) · resource type · date range · search · Export CSV · row Details. Append-only.

Kill Switch — [/kill-switch](#)

Arm (typed KILL) · mode (Halt new entries/Halt all activity/Flatten all positions) · Resume (typed RESUME) · Flatten all. Scopes SYMBOL→ACCOUNT→GLOBAL. Activation history.

Pipeline Health — [/pipeline-health](#)

Env select · component tabs · auto-refresh (5s/15s/30s/1m) · Pause alerts. Per-component status/uptime/latency/throughput/error rate. Pipeline flow diagram, job backlog with Retry.

Risk Decisions — [/risk/decisions](#)

Decision tabs (All/Approved/Rejected/Reduced/Escalated) · rule · symbol · date range · Only-overrides · per-row Override/View. Override modal: reason, justification, size.

Reconciliation — [/reconciliation](#)

Account · date · status tabs · auto-refresh · Run · per-row Investigate/Auto-fix/Mark resolved. Breaks by type, history.

Raw Ingestion Events — [/ingest/events](#)

Symbol · source · stream (Quotes/Trades/Orders/News/Signals/All) · window (15m/1h/6h/24h) · search · auto-refresh · Load more · Copy JSON. Payload expansion.

Part H — Operator Runbook & Safety

| H.1 “Nothing is happening” — diagnostic order

1. Empty `MARKET_DATA_POLL_WATCHLIST`? → the poller no-ops forever. Add symbols.
2. Is it market hours (IST)? Outside market/pre-market the poller pauses.
3. `Provider = mock`? → data is synthetic.
4. Is a worker consuming the stage's queue?
5. Do rules pass the gate? Check `RuleConfig.enabled + validated_regimes[regime].status=="G0"`.
6. `EXECUTION_ENGINE_ENABLED=True`? Otherwise no orders — by design.
7. Check Pipeline Health, Journal, Audit for the trail.

| H.2 Cold start checklist

- ~60 candles per symbol before EMA/ATR/Supertrend rules fire (~1 trading day of 1-min bars). VWAP needs 09:15 IST opening candle.
- Backtest evidence first, review affinities, then enable the specific rules you accept.

| H.3 Safety posture

- Live execution is triple-blocked: `BROKER_ENVIRONMENT=live` fails startup · `ALGO_REGISTRATION_ID=""` refuses live · paper bypass refuses when live. Paper is the only certified mode.
- Risk is fail-closed: kill switches, concentration, freshness, capital, stops.
- Kill Switch page = emergency stop + flatten positions.

Part I — Troubleshooting & Known Limits

Issue	Fix
“Why isn't my stock doing anything?”	Symbol is on the user Watchlist (backtests) but not in <code>MARKET_DATA_POLL_WATCHLIST</code> (live data) nor <code>NEWS_POLL_SYMBOLS</code> (news). Add to the right list.
Backtest job returns ERROR silently	<code>admin</code> user missing/renamed — create it. Check worker log for <code>Owner user 'admin' not found.</code>
Rules never evaluate	Confirm a worker consumes the <code>rule_engine</code> queue.
No broker-connection events	<code>monitor-broker-connection</code> routes to default — ensure consumed.
Webhooks return 501	Set <code>TRADINGVIEW_WEBHOOK_TOKEN</code> / <code>CHARTINK_WEBHOOK_TOKEN</code> .
Rejected orders, code <code>MISSING_STOP_LOSS</code>	By design for rules that don't emit stops; add a stop via risk config or accept the fail-closed block.
Disk fills up (Postgres recovery failures)	Keep ≥ 15 –20% free; prune docker images, clear build caches; back up DB before touching volumes.
Backtest runs very long	Normal for 730-day + heavy-rule combos; safe to interrupt — resumes from cursor.
Affinity panel empty for a symbol	No strategy passed the gate (≥ 30 trades, expectancy > 0 , PF > 1.3 , maxDD $< 20\%$). Correct.
Frontend shows blank/unauthorized after deploy	Check <code>VITE_API_BASE_URL</code> and JWT expiry; the client auto-refreshes tokens, else re-login.

End of manual. TradeVision runs paper trading only as the certified mode. Keep `infra/.env` and `backend/.env` secret and out of git. For environment variables not listed, see `config/settings/base.py` and `infra/.env.example`.